

Agent Skills

Turn General-Purpose AI into Domain Specialists

Geeks Club



February 2026

Agenda

1. 🤔 **What are Skills?** — Definition & mental model
2. 🎯 **Why use Skills?** — Benefits & use cases
3. ⚙️ **How Skills work** — Progressive disclosure & architecture
4. 🏗️ **Skill structure** — Anatomy of a SKILL.md
5. 📐 **Core principles** — Conciseness, degrees of freedom
6. 🖋️ **Creating a Skill** — Step-by-step guide
7. 📦 **Patterns & examples** — Real-world Skills
8. 🚫 **Anti-patterns** — What to avoid
9. 💭 **Discussion**

What are Skills?

Reusable, filesystem-based resources that provide AI agents with **domain-specific expertise**: workflows, context, and best practices.

Key idea:

Skills transform **general-purpose agents** into **specialists**.

Concept	Analogy
 Skill	An onboarding guide for a new team member
 Prompt	A one-off sticky note with instructions
 Tool	A specific capability (e.g., run code, search)

Skills differ from prompts — they **load on-demand** and eliminate the need to repeatedly provide the same guidance

Why Use Skills?

Three key benefits:

-  **Specialize the agent** — Tailor capabilities for domain-specific tasks
-  **Reduce repetition** — Create once, use automatically across conversations
-  **Compose capabilities** — Combine Skills to build complex workflows

Without Skills:

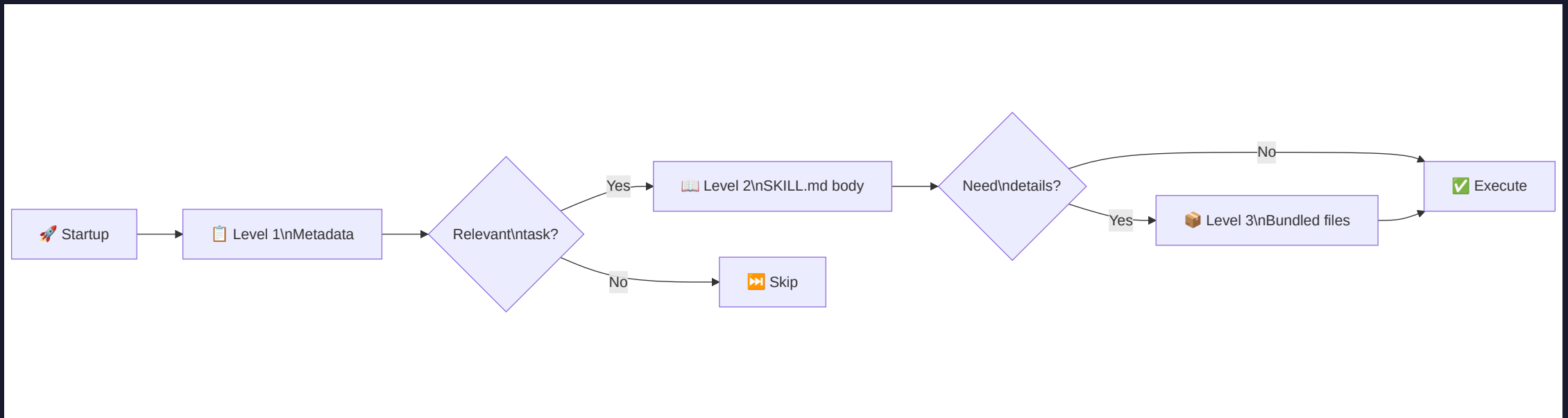
```
You: "Remember to always filter test accounts, use UTC timestamps,  
and follow our naming convention for BigQuery tables..."  
(... every single conversation)
```

With Skills:

```
You: "Analyze Q4 revenue"  
Agent: (automatically loads BigQuery Skill with all your conventions)
```

⚙️ How Skills Work — Progressive Disclosure

Skills leverage a **three-level loading** strategy:



Only relevant content occupies the **context window** at any given time.

Three Levels in Detail

Level	When loaded	Token cost	Content
Level 1 — Metadata	Always (startup)	~100 tokens/Skill	<code>name</code> and <code>description</code> from YAML
Level 2 — Instructions	When triggered	< 5k tokens	SKILL.md body
Level 3 — Resources	As needed	Effectively unlimited	Bundled files, scripts, schemas

Example flow:

User: "Extract text from this PDF"



Agent checks metadata: "pdf-processing" matches! → reads SKILL.md



SKILL.md says: "For forms, see FORMS.md" → user didn't ask about forms → skip



Agent uses instructions from SKILL.md to complete the task



Skill Structure — Anatomy of SKILL.md

Every Skill requires a `SKILL.md` file with **YAML frontmatter**:

```
---
name: pdf-processing
description: Extract text and tables from PDF files, fill forms,
  merge documents. Use when working with PDF files or when the user
  mentions PDFs, forms, or document extraction.
---
```

PDF Processing

Quick start

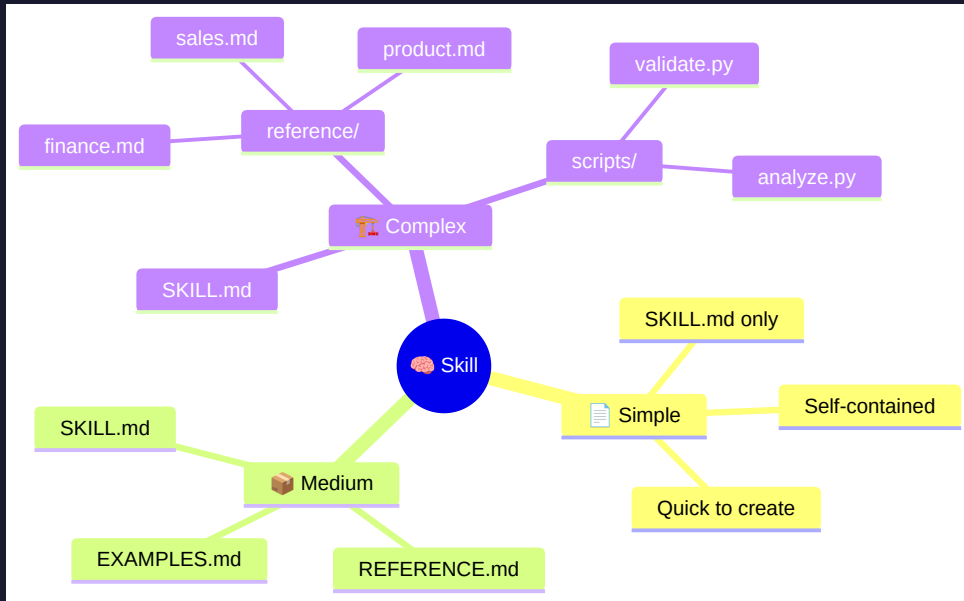
Use pdfplumber to extract text from PDFs:

```
```python
import pdfplumber
with pdfplumber.open("document.pdf") as pdf:
 text = pdf.pages[0].extract_text()
```
```

Advanced features

Directory Structure

A Skill can be simple or complex:



Rule: Keep references one level deep from SKILL.md.

▴ Core Principle 1: Conciseness

The **context window** is a public good. Your Skill shares it with everything else.

Default assumption: The agent is already very smart.

Only add context the agent doesn't already have. Challenge each piece:

| ✗ Too verbose (~150 tokens) | ✓ Concise (~50 tokens) |
|--|--|
| "PDF files are a common format that contains text, images, and other content. To extract text, you'll need a library. There are many libraries available..." | "Use pdfplumber for text extraction:" + code |

Ask yourself:

- 🤔 "Does the agent really **need** this explanation?"

• 🤔 "Can I assume the agent knows this?"

▴ Core Principle 2: Degrees of Freedom

Match **specificity** to the task's fragility:

● High freedom — Multiple valid approaches

Code review process

1. Analyze the code structure and organization
2. Check for potential bugs or edge cases
3. Suggest improvements for readability

● Medium freedom — Preferred pattern with variation

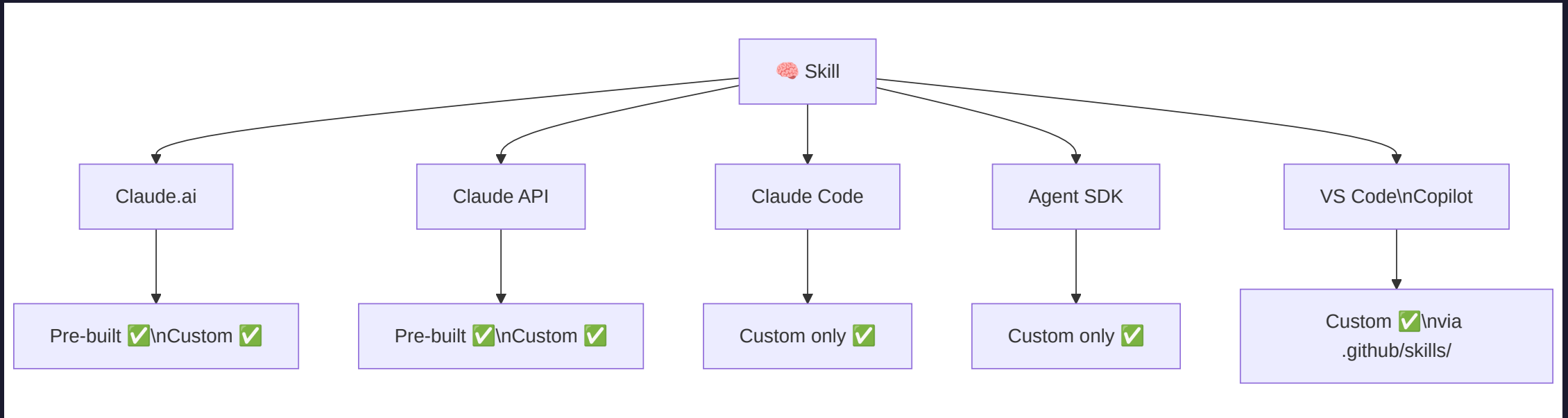
Generate report

Use this template and customize as needed:

```
def generate_report(data, format="markdown", include_charts=True):  
    # Process data, generate output, optionally add visualizations
```

● Low freedom — Fragile, must be exact

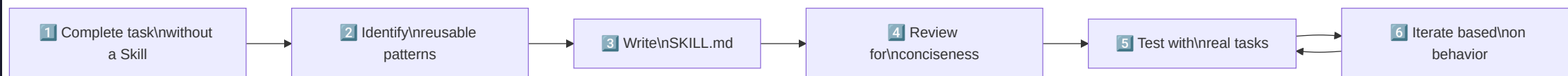
Where Skills Work



 **Skills do NOT sync across surfaces**

Upload/configure separately for each platform.

✍️ Creating a Skill — Step by Step



Step 1: Complete a task with normal prompting first

Notice what context you repeatedly provide.

Step 2: Identify the reusable pattern

Table schemas, naming conventions, filtering rules, query patterns.

✍️ Naming & Description Best Practices

Name format: **gerund form** (verb + -ing)

| ✓ Good names | ✗ Bad names |
|-------------------------------------|---------------------------|
| <code>processing-pdfs</code> | <code>helper</code> |
| <code>analyzing-spreadsheets</code> | <code>utils</code> |
| <code>managing-databases</code> | <code>documents</code> |
| <code>writing-documentation</code> | <code>claude-tools</code> |

Description: **what + when** in third person

```
# ✓ Good
description: "Analyze Excel spreadsheets, create pivot tables,
generate charts. Use when analyzing Excel files, spreadsheets,
tabular data, or .xlsx files."
```

Pattern: Progressive Disclosure

Pattern 1 — High-level guide with references:

```
SKILL.md      → Quick start + links to details
FORMS.md     → Form-filling guide (loaded as needed)
REFERENCE.md  → API reference (loaded as needed)
```

Pattern 2 — Domain-specific organization:

```
bigquery-skill/
├── SKILL.md (overview and navigation)
├── reference/
│   ├── finance.md    (revenue, billing metrics)
│   ├── sales.md      (opportunities, pipeline)
│   └── product.md    (API usage, features)
```

Pattern 3 — Conditional details:

Workflows & Feedback Loops

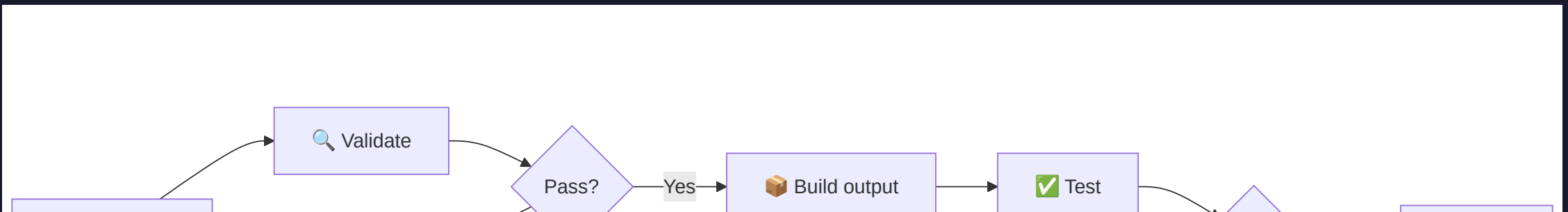
Use workflows for complex tasks:

PDF form filling workflow

Task Progress:

- [] Step 1: Analyze the form (run `analyze_form.py`)
- [] Step 2: Create field mapping (edit `fields.json`)
- [] Step 3: Validate mapping (run `validate_fields.py`)
- [] Step 4: Fill the form (run `fill_form.py`)
- [] Step 5: Verify output (run `verify_output.py`)

Implement feedback loops:



Example: Our Presentation Skill

We use a Skill right here in this repo!

```
.github/skills/creating-geeks-club-marp-presentation/  
└── SKILL.md
```

```
name: creating-geeks-club-marp-presentation  
description: Creates Marp slide deck presentations for Geeks Club –  
a developer-to-developer knowledge sharing meeting. Use when the  
user asks to create, scaffold, or generate a new presentation,  
slide deck, or talk for Geeks Club.
```

What it encodes:

-  **Visual identity** — dark theme, color palette, CSS
-  **Folder structure** — numbered folders, naming conventions
-  **Diagram standards** — Mermaid syntax. max 8 nodes

Example: BigQuery Analytics Skill

```
bigquery-skill/  
├── SKILL.md  
└── reference/  
    ├── finance.md      (revenue metrics, ARR, billing)  
    ├── sales.md        (opportunities, pipeline, accounts)  
    ├── product.md      (API usage, features, adoption)  
    └── marketing.md    (campaigns, attribution, email)
```

What SKILL.md contains:

Rules

- ALWAYS filter out test accounts (`account_type != 'test'`)
- Use UTC timestamps for all date comparisons
- Follow naming convention: `{domain}_{metric}_{period}`

Quick search

Find specific metrics:

```
grep -i "revenue" reference/finance.md
```

Example: Code with Utility Scripts

Pre-made scripts > Generated code

```
pdf-skill/  
├── SKILL.md  
├── scripts/  
│   ├── analyze_form.py    → Extract form fields  
│   ├── validate_fields.py → Check for errors  
│   └── fill_form.py       → Apply values to PDF
```

Why scripts are better than asking the agent to write code:

| Aspect | Script | Generated code |
|---|-------------------|----------------------|
|  Reliability | Battle-tested | May have bugs |
|  Token cost | Output only | Full code in context |
|  Speed | Instant execution | Generation time |
|  | | |

🚫 Anti-Patterns to Avoid

✗ Over-explaining what the agent already knows

```
# Bad: "PDF files are a common format..."  
# Good: "Use pdfplumber for text extraction:"
```

✗ Offering too many options

```
# Bad: "You can use pypdf, or pdfplumber, or PyMuPDF, or pdf2image..."  
# Good: "Use pdfplumber. For scanned PDFs requiring OCR, use pdf2image."
```

✗ Deeply nested references

```
# Bad: SKILL.md → advanced.md → details.md → actual info  
# Good: SKILL.md → advanced.md (one level deep)
```

✗ Windows-style paths

Security Considerations

 Use Skills **only from trusted sources** — those you created yourself or from Anthropic.

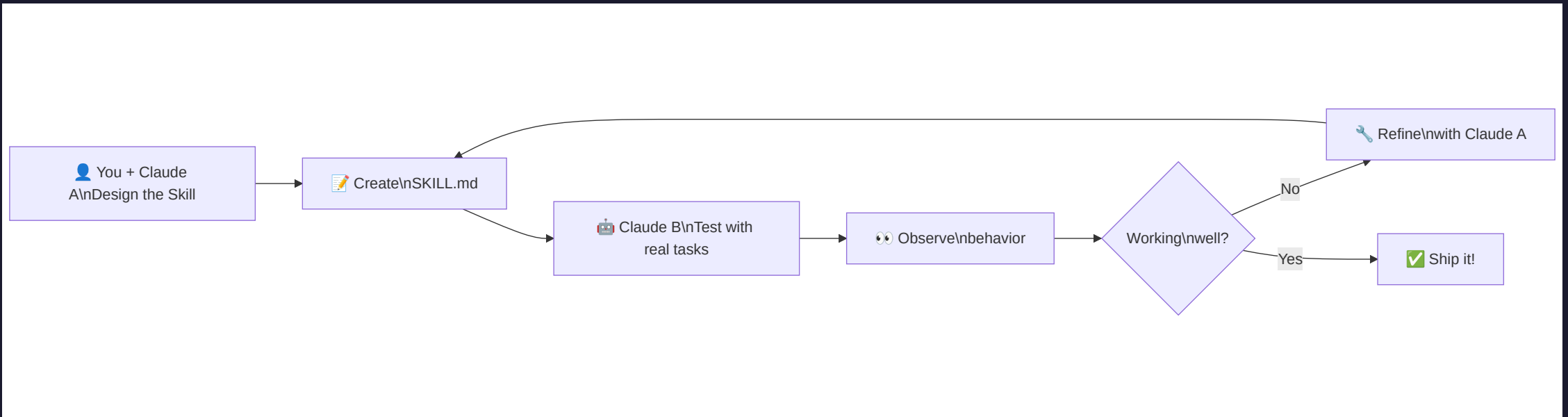
Risks of untrusted Skills:

-  **Data exfiltration** — leaking sensitive info to external systems
-  **Unauthorized access** — misusing file operations and bash commands
-  **Hidden behavior** — instructions that don't match stated purpose
-  **External dependencies** — fetched content may contain malicious instructions

Checklist before using third-party Skills:

- [] Review ALL files: SKILL.md, scripts, resources

Iterative Development



Two-agent pattern:

- **Claude A** (expert) — helps you design and refine the Skill
- **Claude B** (tester) — uses the Skill on real tasks, reveals gaps

Key observations to track:

✓ Checklist for Effective Skills

Core quality:

- [] Description is **specific** and includes trigger keywords
- [] SKILL.md body is **under 500 lines**
- [] **No time-sensitive** information
- [] **Consistent terminology** throughout
- [] **Concrete examples**, not abstract descriptions
- [] References are **one level deep**

Code and scripts:

- [] Scripts **solve problems** rather than punt to the agent
- [] Error handling is **explicit and helpful**

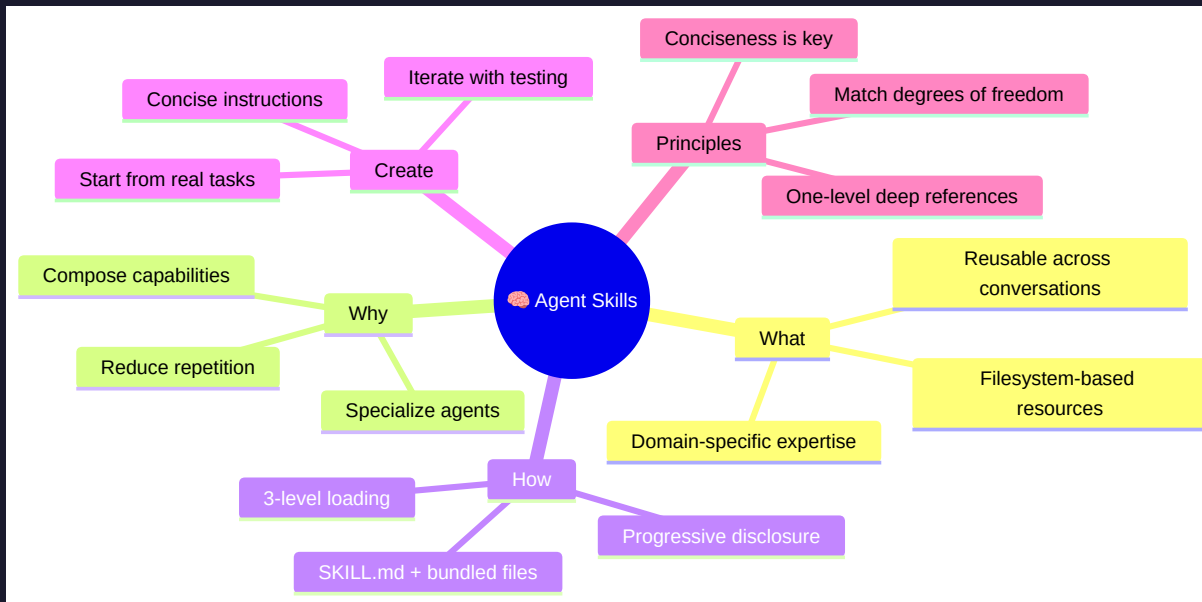
Key Takeaways

1.  **Skills = persistent domain knowledge** for AI agents
Create once, use automatically across conversations
2.  **Progressive disclosure** is the core architecture
Only relevant content enters the context window
3.  **Conciseness is king** — the agent is already smart
Only provide what it can't infer on its own
4.  **Iterate based on observation**, not assumptions
Watch how the agent uses your Skill, then refine
5.  **Start simple** — a single SKILL.md can be very powerful
Add complexity only when needed

Discussion

1. 🤔 What **repetitive context** do you provide to AI agents that could become a Skill?
2. 🏢 How could we use Skills for **team-wide conventions** (coding standards, PR reviews, deployment procedures)?
3. 🔒 How should we handle **security review** for shared Skills in our organization?
4. 🧩 Could we build a **library of Skills** for common tasks across our projects?
5. ⚖️ Where do you draw the line between **a Skill** and **a prompt template**?

Summary





Sources

- [Agent Skills Overview](#) — Official documentation
- [Skill Authoring Best Practices](#) — Core principles & patterns
- [Skills Quickstart](#) — Getting started tutorial
- [Agent Skills Cookbook](#) — Hands-on examples
- [Equipping Agents for the Real World](#) — Engineering deep-dive blog
- [Use Skills in Claude Code](#) — Claude Code integration
- [Use Skills with the API](#) — API integration guide

 **Thank You!**

Questions?

Geeks Club — February 2026

"Skills transform general-purpose agents into specialists."